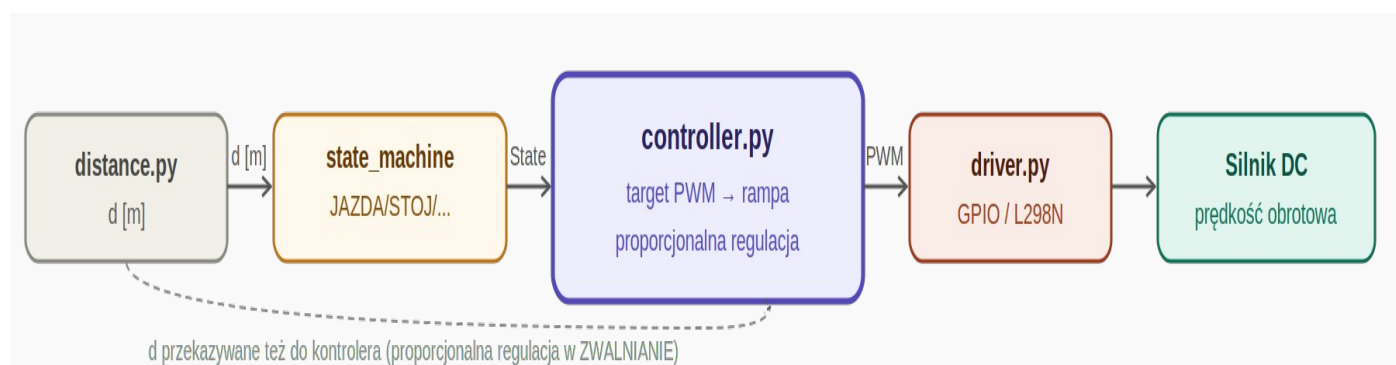


controller.py — teoria i opis modułu sterowania prędkością

1. Rola modułu w systemie Stop & Go

Moduł `controller.py` zajmuje ostatnie ogniwo w łańcuchu decyzyjnym przed sprzętem. Przyjmuje dwa wejścia: aktualny stan z maszyny stanów (JAZDA / ZWALNIANIE / STOJ / RUSZANIE) oraz odległość d w metrach z `distance.py`. Na ich podstawie oblicza wartość PWM z przedziału $[0.0, 1.0]$ którą przekazuje do `driver.py`. Ta jedna liczba określa prędkość silnika napędowego.

Moduł realizuje dwie odrębne funkcje regulacyjne: mapowanie stan $\rightarrow$ wartość docelowa PWM (w tym proporcjonalną regulację prędkości w stanie ZWALNIANIE) oraz ogranicznik szybkości zmian PWM (slew rate limiter, rampa). Pierwsza funkcja odpowiada na pytanie 'jak szybko chcemy jechać', druga na pytanie 'jak szybko możemy zmienić prędkość'.



Rysunek 1. Miejsce kontrolera w łańcuchu przetwarzania. Kontroler łączy wyjście maszyny stanów z warstwą sprzętową `driver.py`.

2. Sygnał PWM — modulacja szerokości impulsu

PWM (Pulse Width Modulation — modulacja szerokości impulsu) to metoda sterowania prędkością silnika DC polegająca na bardzo szybkim włączaniu i wyłączaniu napięcia zasilania. Silnik DC nie reaguje na każde włączenie/wyłączenie osobno — jego bezwładność mechaniczna (i elektryczna — indukcyjność uzwojeń) uśrednia napięcie. Efektem jest napięcie efektywne proporcjonalne do wypełnienia sygnału PWM.

```
Wypełnienie PWM = 0.0 (0%) → napięcie efektywne = 0 V → silnik stoi
Wypełnienie PWM = 0.5 (50%) → napięcie efektywne = 6 V → połowa prędkości (przy 12V)
Wypełnienie PWM = 1.0 (100%) → napięcie efektywne = 12 V → pełna prędkość
```

W systemie używamy PWM jako abstrakcji — kontroler operuje na wartościach $[0.0, 1.0]$ reprezentujących procent maksymalnej mocy silnika. Driver.py przelicza tę wartość na konkretny sygnał sprzętowy (np. `ChangeDutyCycle(pwm * 100)` dla `RPi.GPIO.PWM`). Dzięki tej abstrakcji kontroler jest niezależny od napięcia zasilania, parametrów silnika i sprzętu.

Częstotliwość PWM a wydajność silnika:

`RPi.GPIO.PWM` używa domyślnie 1 kHz — czyli 1000 cykli na sekundę.
Przy tej częstotliwości silnik DC pracuje płynnie bez słyszalnego bzyczenia.
Poniżej ~100 Hz silnik może wydawać charakterystyczny dźwięk (wibracje mechaniczne przy niskim wypełnieniu). Powyżej ~20 kHz (ultradźwięki) praca jest całkowicie cicha ale sterownik L298N może się przegrzewać.
1 kHz to optymalny kompromis dla modelu RC.

3. Stałe konfiguracyjne — uzasadnienie wartości

<code>V_SET = 0.55</code>	Nominalna prędkość jazdy swobodnej — 55% mocy. Nie używamy 100% aby zostawić margines termiczny silnika i pełne pasmo regulacji poniżej <code>V_SET</code> dla stanu ZWALNIANIE.
<code>V_MIN = 0.20</code>	Minimalne PWM przy którym silnik DC pokonuje tarcie statyczne i obraca kołami. Wartość eksperymentalna — zależy od konkretnego silnika, przekładni i podłoża. Dolna granica regulacji proporcjonalnej.
<code>V_START = 0.25</code>	Prędkość startowa w stanie RUSZANIE. Wyższa niż <code>V_MIN</code> bo przy ruszaniu silnik musi pokonać bezwładność pojazdu ($F=ma$) oprócz tarcia statycznego.
<code>RAMP_UP = 0.40</code>	Maksymalna szybkość wzrostu PWM [jednostki/s]. Czas przyspieszenia od 0 do <code>V_SET</code> : $0.55/0.40 = 1.375$ s. Chroni silnik przed prądem rozruchowym.
<code>RAMP_DOWN = 0.80</code>	Maksymalna szybkość spadku PWM [jednostki/s]. Czas hamowania od <code>V_SET</code> do 0: $0.55/0.80 = 0.69$ s. Prawie 2× szybsze niż przyspieszanie — zasada: hamowanie zawsze szybsze.

Prąd rozruchowy silnika DC — dlaczego `RAMP_UP` jest konieczny?

Silnik DC w chwili startu ($n=0$) ma impedancję równą samej rezystancji uzwojeń — reaktancja indukcyjna jest zerowa (bo di/dt jest duże tylko przez ułamek sekundy). Prąd rozruchowy $I = U/R$ może być 5–10× większy niż prąd nominalny.

Dla silnika 1A nominalny, $U=6V$, $R=1\Omega$:

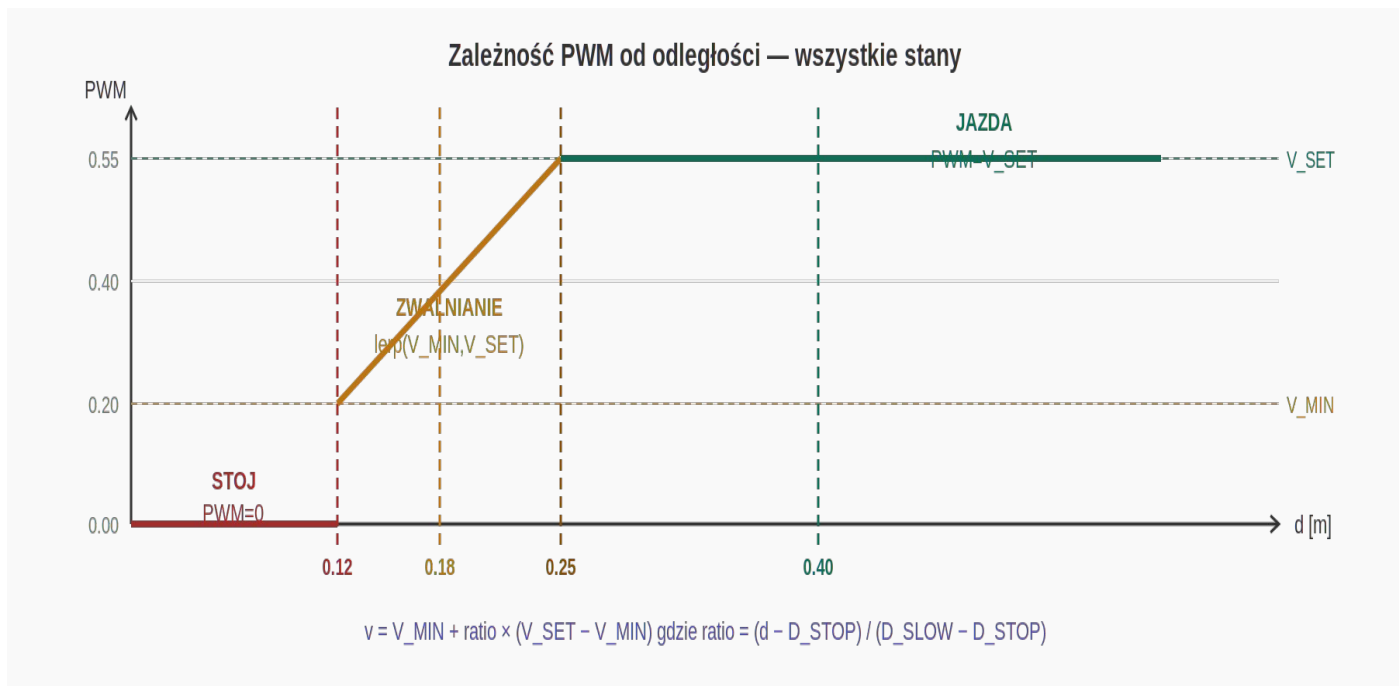
Prąd rozruchowy: $I = 6/1 = 6A$ (6× przekroczenie normy)

Raspberry Pi 4 może dostarczyć łącznie ~3A. Nagłe pobranie 6A przez silnik powoduje spadek napięcia na szynie 5V i reset RPi. Rampa `RAMP_UP=0.40` rozciąga czas startu do 1.375 s — prąd rozruchowy jest ograniczony przez stopniowy wzrost wypełnienia PWM.

4. Proporcjonalna regulacja prędkości w stanie ZWALNIANIE

Metoda `_proportional_speed()` implementuje człon proporcjonalny (P) regulatora PID dla stanu ZWALNIANIE. Jest to najprostszy, a zarazem jeden z najczęściej stosowanych algorytmów regulacji — sterowanie jest proporcjonalne do uchybu (różnicy między aktualną a zadaną wartością).

W naszym przypadku uchybem jest odległość: im bliżej poprzednika (mniejsze d), tym mniejsza zadana prędkość. Odległość d jest normalizowana do bezwymiarowego ratio w przedziale $[0, 1]$, a następnie przekształcana na PWM przez interpolację liniową między `V_MIN` a `V_SET`.



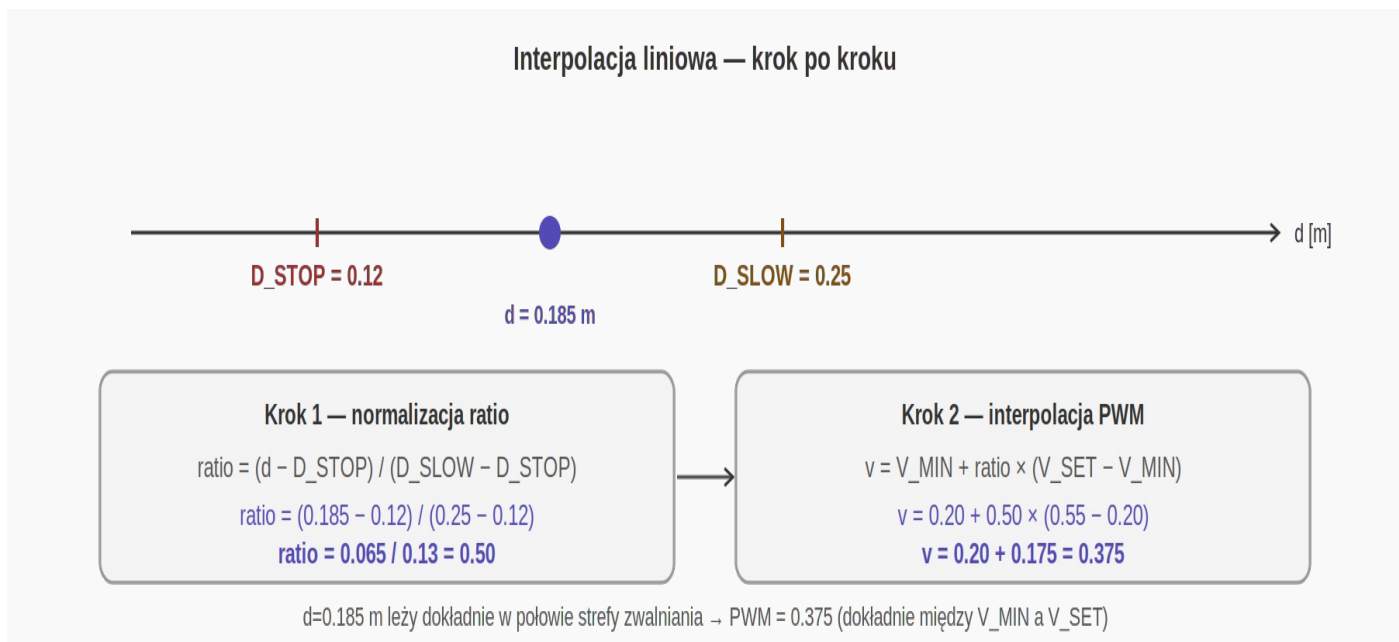
Rysunek 2. Zależność PWM od odległości dla wszystkich stanów. Strefa ZWALNIANIE implementuje liniową interpolację między V_MIN (przy D_STOP) a V_SET (przy D_SLOW i powyżej).

4.1 Matematyka interpolacji liniowej (lerp)

Interpolacja liniowa (linear interpolation, lerp) to operacja wyznaczenia wartości pośredniej między dwoma końcowymi wartościami na podstawie parametru $t \in [0, 1]$. Ogólna formuła:

$$\text{lerp}(a, b, t) = a + t \cdot (b - a)$$

gdzie: a = wartość przy $t=0$ ($V_MIN = 0.20$ przy $d = D_STOP$)
 b = wartość przy $t=1$ ($V_SET = 0.55$ przy $d = D_SLOW$)
 $t = \text{ratio} = (d - D_STOP) / (D_SLOW - D_STOP)$



Rysunek 3. Przykład interpolacji liniowej dla $d=0.185 \text{ m}$ — dokładnie w środku strefy zwalniania. $\text{ratio}=0.50$ daje $\text{PWM}=0.375$, czyli połowę drogi między V_MIN a V_SET .

4.2 Saturacja (clipping) — ochrona przed wartościami spoza zakresu

Linia $\text{ratio} = \max(0.0, \min(1.0, \text{ratio}))$ to implementacja saturacji — nieliniowego elementu ograniczającego wyjście do zdefiniowanego zakresu. Jest to standardowy element w każdym regulatorze przemysłowym.

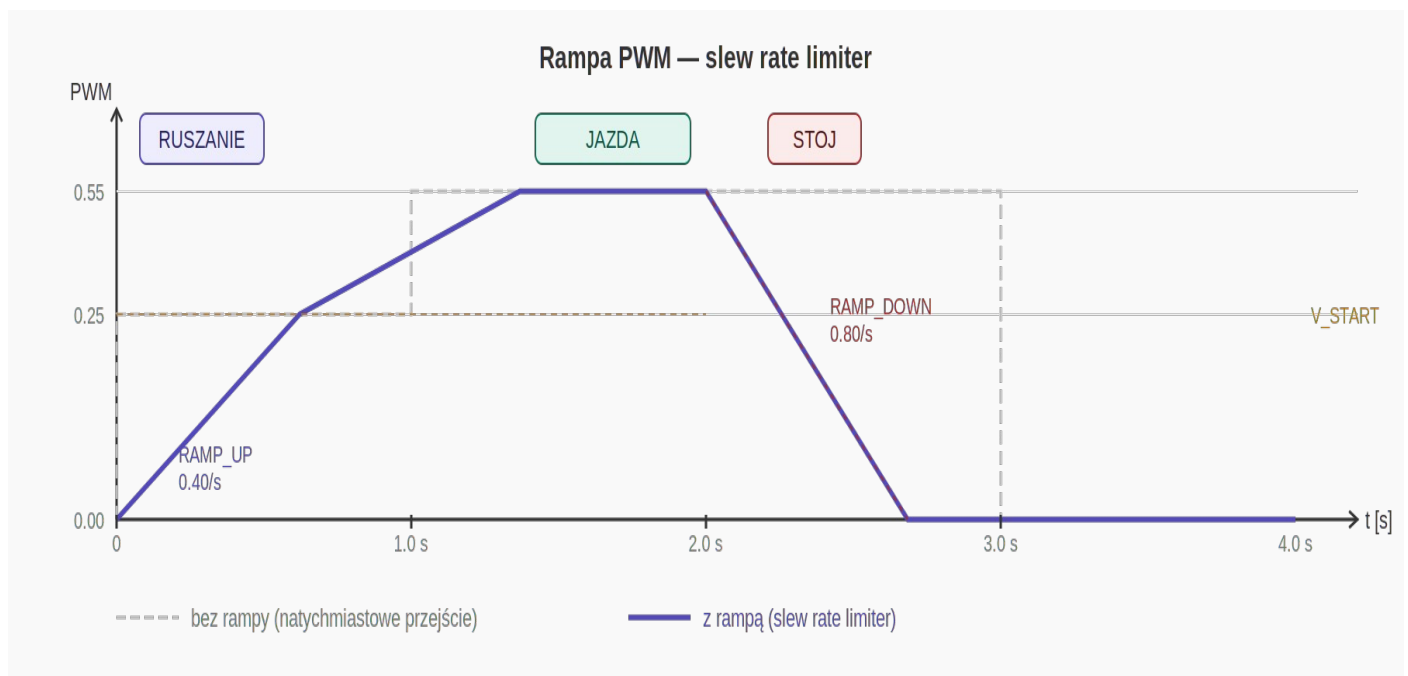
```
d = 0.08 m (poniżej D_STOP): ratio = (0.08-0.12)/0.13 = -0.31 → po clip: 0.0
d = 0.30 m (powyżej D_SLOW): ratio = (0.30-0.12)/0.13 = 1.38 → po clip: 1.0
d = 0.185 m (w środku): ratio = (0.185-0.12)/0.13 = 0.50 → bez zmiany
```

Bez saturacji wartości spoza zakresu dałyby PWM poniżej V_MIN (silnik nie reaguje) lub powyżej V_SET (przekraczamy nominalną prędkość jazdy). Saturacja gwarantuje że wynik zawsze mieści się w sensownym fizycznie przedziale $[V_MIN, V_SET]$.

5. Slew rate limiter — rampa PWM

Slew rate limiter (ogranicznik szybkości zmian) to element układów sterowania który ogranicza maksymalną szybkość zmiany sygnału wyjściowego w czasie. Jest powszechnie stosowany w elektronice (wzmacniacze operacyjne, przetwornice DC/DC), robotyce (profile ruchu ramion robotycznych) i systemach ADAS (ograniczenie szarpnięć podczas hamowania).

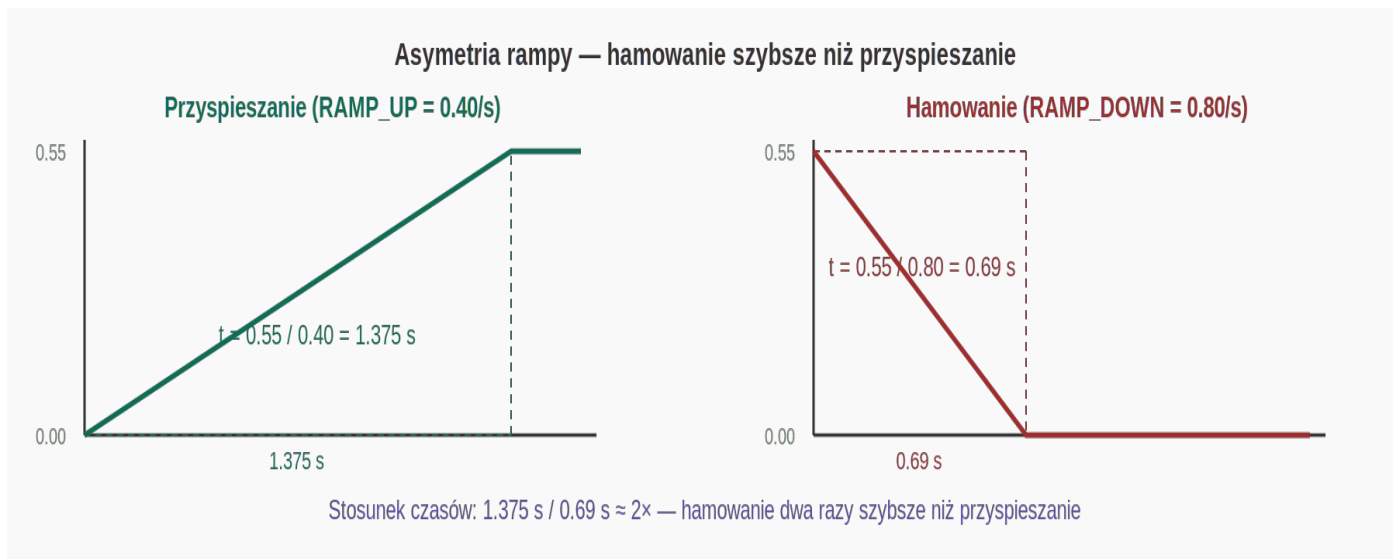
W naszym systemie rampa ogranicza szybkość zmian PWM do $RAMP_UP$ lub $RAMP_DOWN$ jednostek na sekundę. Obliczenia są oparte na rzeczywistym czasie Δt między wywołaniami — nie na nominalnej częstotliwości pętli. Dzięki temu rampa jest fizycznie dokładna niezależnie od obciążenia procesora.



Rysunek 4. Porównanie odpowiedzi PWM bez rampy (linia przerywana) i z rampą (linia ciągła). Rampa eliminuje nagłe skoki które powodowałyby przeciążenie silnika.

5.1 Asymetria $RAMP_UP$ vs $RAMP_DOWN$

Kluczowym elementem projektu jest asymetria rampy: hamowanie ($RAMP_DOWN = 0.80$) jest prawie dwa razy szybsze niż przyspieszanie ($RAMP_UP = 0.40$). Ta asymetria wynika z zasady bezpieczeństwa obowiązującej we wszystkich systemach ADAS: w sytuacji zagrożenia pojazd musi reagować natychmiast.



Rysunek 5. Asymetria czasów przyspieszania i hamowania. Hamowanie zajmuje 0.69 s, przyspieszanie 1.375 s — stosunek 2:1 jest świadomą decyzją projektową.

Fizyczne uzasadnienie asymetrii:

Przyspieszanie: ograniczone przez prąd rozruchowy silnika i komfort pasażera.

Zbyt szybkie → reset RPi z powodu overcurrent, szarpnięcie mechaniczne.

Hamowanie: ograniczone tylko przez bezpieczeństwo — musi być jak najszybsze.

Szybkie hamowanie = mniejsza przestrzeń zatrzymania = mniejsze ryzyko kolizji.

W normie ISO 15622 (ACC) wymagane jest by czas reakcji układu hamowania był nie dłuższy niż 0.6 s od momentu detekcji zagrożenia. Nasz RAMP_DOWN przy pełnej prędkości $V_SET=0.55$ daje czas zatrzymania 0.69 s — bliski normie.

5.2 Implementacja rampy — analiza matematyczna

```
def _ramp(self, current, target, dt):
    if target > current:
        max_step = RAMP_UP * dt      # max wzrost w tej klatce
    else:
        max_step = RAMP_DOWN * dt    # max spadek w tej klatce

    diff = target - current          # jak daleko do celu
    if abs(diff) <= max_step:
        return target                # jesteśmy blisko — skocz do celu
    return current + max_step * sign(diff) # ogranicz krok
```

Warunek $\text{abs}(\text{diff}) \leq \text{max_step}$ sprawdza czy możemy osiągnąć cel w tej klatce. Jeśli tak — ustawiamy dokładnie target, bez przekraczania (overshoot). To ważne: bez tego warunku przy małych diff system oscylowałby wokół target zamiast do niego dążyć.

Użycie dt (rzeczywistego czasu klatki) zamiast nominalnego 1/30 s sprawia że rampa jest odporna na jitter pętli głównej. Gdy detekcja zajęła 80 ms zamiast 33 ms, $\text{dt}=0.080$, $\text{max_step}=0.032$ — rampa automatycznie kompensuje opóźnienie.

6. Rzeczywisty czas dt — dlaczego nie używamy nominalnego 1/FPS

Jednym z subtelniejszych elementów kontrolera jest użycie rzeczywistego czasu między wywołaniami ($\text{dt} = \text{time.time() - last_time}$) zamiast nominalnej częstotliwości pętli. To pozornie mała różnica ma duże znaczenie w praktyce.

Scenariusz: pętla nominalna 30 fps = dt_nom = 0.0333 s

Klatka 1: detekcja HOG zajmuje 150 ms → rzeczywiste dt = 0.150 s

Bez rzeczywistego dt: max_step = 0.40 * 0.033 = 0.0132 (za mały krok)

Z rzeczywistym dt: max_step = 0.40 * 0.150 = 0.060 (poprawny krok)

Klatka 2: tracker zajmuje 5 ms → dt = 0.005 s

Bez rzeczywistego dt: max_step = 0.40 * 0.033 = 0.0132 (za duży krok!)

Z rzeczywistym dt: max_step = 0.40 * 0.005 = 0.002 (poprawny krok)

Bez rzeczywistego dt rampa jest nieregularna — przy ciężkich klatkach (HOG) zmiany PWM są za małe, przy lekkich (tracker) za duże. Z rzeczywistym dt cała zmian PWM w czasie jest stała niezależnie od rozkładu obciążenia procesora.

Całka zmian PWM = praca silnika:

Pole pod krzywą PWM(t) jest proporcjonalne do energii dostarczonej do silnika.

Stały RAMP_UP * dt zapewnia że ta energia rośnie liniowo w czasie — pojazd przyspiesza jednostajnie niezależnie od zmiennego obciążenia procesora.

To jest fizyczne uzasadnienie dla dt — nie tylko elegancja kodu.

7. Wzorzec Dependency Injection — parametry d_slow i d_stop

Konstruktor kontrolera przyjmuje progi d_slow i d_stop jako parametry zamiast używać stałych z StopGoStateMachine bezpośrednio. Jest to implementacja wzorca Dependency Injection (wstrzykiwanie zależności) — jednego z fundamentalnych wzorców projektowych w inżynierii oprogramowania.

Zasada: moduł nie tworzy swoich zależności sam — otrzymuje je z zewnątrz. W main.py wygląda to tak:

```
# main.py - zależności wstrzykiwane z zewnątrz
ctrl = DriveController(
    d_slow = StopGoStateMachine.D_SLOW,    # 0.25
    d_stop = StopGoStateMachine.D_STOP,    # 0.12
)

# Gdyby kontroler importował StopGoStateMachine bezpośrednio:
# - zmiana progu D_SLOW wymagałaby modyfikacji controller.py
# - testowanie kontrolera wymagałoby instancji StopGoStateMachine
# - oba moduły byłyby ściśle sprzężone (tight coupling)
```

Dzięki Dependency Injection kontroler można testować z dowolnymi progami bez modyfikowania kodu. Można też podmienić maszyny stanów na inną implementację (np. uproszczoną do testów) — kontroler pozostaje niezmieniony dopóki progi są przekazywane poprawnie.

8. Interfejs publiczny

<code>update(state, d)</code>	Główna metoda. Przyjmuje stan i odległość, zwraca PWM ∈ [0.0, 1.0]. Wywołać raz na każdą klatkę — ta sama częstotliwość co maszyna stanów.
<code>pwm</code>	Property (odczyt). Aktualnie wysterowane PWM po rampie. Przydatne do logowania i debugowania bez zapamiętywania wyniku update().

9. Odniesienia literaturowe

1. Kaehler A., Bradski G. — Learning OpenCV 3, O'Reilly 2016.

Kontekst systemu wizyjnego dostarczającego dane wejściowe do kontrolera.

2. Shirpour M. et al. — Traffic Object Detection Based on the Attentional Visual Field of Drivers. IEEE Transactions on Intelligent Vehicles, 2023.

Wymagania systemów ADAS dotyczące czasów reakcji i charakterystyki hamowania.

3. Åström K.J., Wittenmark B. — Computer-Controlled Systems: Theory and Design. Prentice Hall, 1997.

Rozdział 3: Regulatory proporcjonalne (P) i człon saturacji. Rozdział 8: Rate limiters i implementacja dyskretna.

4. ISO 15622:2018 — Intelligent transport systems — Adaptive cruise control systems.

Wymagania dotyczące profili przyspieszania i hamowania w systemach ACC (rozdz. 5.3 i 6.2).

5. Dokumentacja wybranego komputera jednopłytkowego (Raspberry Pi / Jetson Nano).

Specyfikacja PWM GPIO: częstotliwość, rozdzielczość duty cycle, prąd maksymalny pinów.